Глава 1. Теоретическая часть

1.1. Персистентные структуры данных: постановка задачи

Большинство классических структур данных являются *эфемерными* (ephemeral): каждая операция изменения разрушает предыдущее состояние, и после выполнения операции доступна только текущая версия структуры. Во многих задачах этого недостаточно: требуется иметь возможность вернуться к любому прошлому состоянию, сравнить несколько состояний между собой или продолжить работу из старого состояния, не потеряв новое.

Структура данных называется *персистентной* (persistent, «долговременной»), если она сохраняет все свои предыдущие версии и позволяет обращаться к ним после выполнения изменений. Термин и систематическая теория персистентности были предложены в работе Дж. Дрисколла, Н. Сарнака, Д. Слейтора и Р. Тарьяна (1989). В зависимости от того, какие операции разрешены над старыми версиями, выделяют несколько уровней персистентности:

- **частичная персистентность** (partial) — все версии доступны для чтения, но изменять можно только последнюю; версии образуют линейную последовательность;
- **полная персистентность** (full) — любую версию можно и читать, и изменять; изменение старой версии порождает новую ветку, и версии образуют дерево;
- **конфлюэнтная персистентность** (confluent) — дополнительно разрешены операции, объединяющие две версии в одну (например, конкатенация); версии образуют ориентированный ациклический граф;
- **функциональная персистентность** — частный случай, при котором узлы структуры после создания вообще никогда не изменяются (так устроены структуры данных в функциональных языках — Haskell, OCaml, Clojure).

Формально задача ставится так. Пусть над структурой выполняется последовательность операций, и после i-й операции получается версия v_i. Требуется реализовать структуру, которая:

1. по номеру версии v позволяет выполнить любую операцию чтения над состоянием v;
2. по номеру версии v позволяет выполнить операцию изменения и получить новую версию v', не изменив v;
3. тратит на каждую операцию время и память, сравнимые с затратами эфемерной структуры.

Наивное решение — копировать всю структуру перед каждым изменением — удовлетворяет первым двум требованиям, но нарушает третье: при n элементах каждая операция стоит $\Theta(n)$ времени и памяти. В данной работе рассматривается персистентный стек — структура, для которой требование эффективности выполняется идеально: все операции над любой версией выполняются за $O(1)$.

1.2. Стек как абстрактный тип данных

Стек (stack) — абстрактный тип данных, хранящий последовательность элементов и работающий по принципу LIFO (Last In — First Out, «последним пришёл — первым ушёл»): добавлять и удалять элементы можно только с одного конца, называемого вершиной (top). Стек поддерживает операции:

Операция	Описание	Эфемерный стек
push(x)	положить элемент x на вершину	O(1) (амортизированно для массива)
pop()	удалить элемент с вершины	O(1)
top()	вернуть элемент на вершине, не удаляя его	O(1)
size() / empty()	число элементов / проверка на пустоту	O(1)

Эфемерный стек обычно реализуется динамическим массивом или односвязным списком. В обоих случаях операция pop физически уничтожает информацию о верхнем элементе (затирает ячейку массива или освобождает узел списка), поэтому вернуться к состоянию до pop невозможно без внешнего журнала операций.

В персистентном стеке каждая операция получает дополнительный аргумент — номер версии, к которой она применяется, а операции изменения возвращают номер новой версии:

```
push(v, x) -> v'     // новая версия: стек версии v с элементом x на вершине
pop(v)     -> v'     // новая версия: стек версии v без верхнего элемента
top(v)     -> x      // элемент на вершине версии v
size(v)    -> k      // число элементов в версии v
```

Версия v при этом остаётся неизменной и доступной для дальнейших операций.

1.3. Структура персистентного стека

Ключевое наблюдение: односвязный список, в котором узлы никогда не изменяются после создания, уже является полностью персистентным стеком. Операция push не трогает существующие узлы — она лишь создаёт новый узел, который ссылается на старую вершину. Операция pop вообще не создаёт и не изменяет узлов — новой версией становится ссылка на следующий узел. Таким образом, разные версии *разделяют общие части* (structural sharing), а копирование не требуется вовсе.

Персистентный стек состоит из двух частей.

Узлы — неизменяемые записи вида (value, next, size), где value — хранимый элемент, next — ссылка на узел, лежащий под ним (или null для дна стека), size — число элементов в стеке, вершиной которого является данный узел. Поле size вычисляется один раз при создании: size = size(next) + 1.

Массив версий roots — для каждой версии хранится ссылка на узел-вершину. Версия 0 — пустой стек (roots[0] = null). Номер новой версии — это просто следующий индекс в массиве roots.

```
class Node:
    value, next, size          # поля не изменяются после создания

roots = [null]                 # версия 0 — пустой стек

push(v, x):
    node = Node(value = x, next = roots[v], size = size(v) + 1)
    roots.append(node)
    вернуть len(roots) - 1      # номер новой версии

pop(v):
```

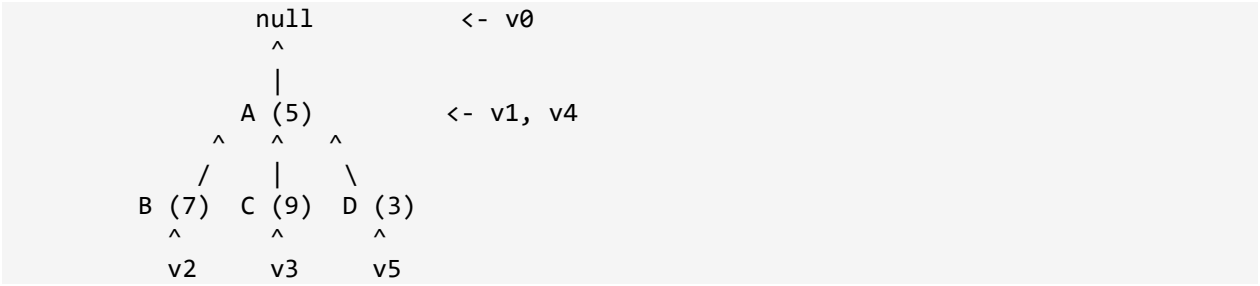
```
    если roots[v] == null: ошибка «стек пуст»
    roots.append(roots[v].next) # новый узел НЕ создаётся
    вернуть len(roots) - 1

top(v):  вернуть roots[v].value
size(v): вернуть 0, если roots[v] == null, иначе roots[v].size
```

Пример. Выполним последовательность операций (справа — содержимое версии от вершины к дну):

```
v0 = пустой стек                    -> []
v1 = push(v0, 5)                    -> [5]
v2 = push(v1, 7)                    -> [7, 5]
v3 = push(v1, 9)    // ветвление!   -> [9, 5]
v4 = pop(v2)                        -> [5]
v5 = push(v4, 3)                    -> [3, 5]
```

Будет создано всего четыре узла: A(5), B(7), C(9), D(3), причём узел A разделяется между всеми непустыми версиями. Связи между узлами образуют дерево, корнем которого служит «пустой стек» (null):



Содержимое любой версии — это путь от её узла-вершины к корню дерева. Версии v1 и v4 совпадают по содержимому и указывают на один и тот же узел, а ветвление версий (v2, v3 и v5 получены из одного и того же состояния [5]) не потребовало копирования ни одного элемента.

1.4. Свойства и инварианты

Корректность персистентного стека опирается на следующие инварианты.

1. **Неизменяемость узлов.** После создания поля value, next и size узла никогда не изменяются. Именно это гарантирует, что операция над одной версией не может повлиять на другую.
2. **Ссылки направлены «в прошлое».** Поле next нового узла указывает на узел, созданный раньше него (или на null). Следовательно, в графе узлов нет циклов, и он является деревом с корнем null (деревом версий).
3. **Соответствие версий и путей.** Содержимое версии v, перечисленное от вершины к дну, совпадает с последовательностью значений на пути от roots[v] до корня.
4. **Корректность размера.** Для любого узла u выполнено size(u) = size(u.next) + 1, size(null) = 0, т. е. size(u) равно глубине узла u в дереве версий.
5. **Монотонность массива версий.** Элементы массива roots только добавляются в конец и никогда не перезаписываются; поэтому номер версии, однажды выданный пользователю, остаётся действительным навсегда.

Утверждение. Для любой последовательности операций и любой версии v результат top(v), size(v) и содержимое версии совпадают с результатом, который дал бы эфемерный стек, если применить к нему цепочку операций, приведшую к версии v.

Доказательство проводится индукцией по номеру версии. База: версия 0 — пустой стек, roots[0] = null, путь до корня пуст. Шаг: пусть утверждение верно для всех версий с номерами меньше k. Если версия k получена операцией push(v, x), v < k, то её путь — это новый узел со значением x, за которым следует путь версии v; по предположению индукции путь v описывает правильный стек, значит, путь k описывает тот же стек с x на вершине — что и требуется от push. Если версия k получена операцией pop(v), то её путь — путь версии v без первого узла, т. е. стек версии v без вершины. Поскольку по инварианту 1 узлы пути версии v не менялись, применение операции к версии v не затронуло ни одной другой версии. ■

1.5. Общие методы обеспечения персистентности

Чтобы оценить место персистентного стека среди других решений, рассмотрим общие методы, которые позволяют сделать персистентной произвольную связную структуру данных.

- **Полное копирование** (full copy). Перед каждым изменением структура копируется целиком. Метод универсален, но требует $\Theta(n)$ времени и памяти на операцию.
- **Копирование пути** (path copying). Копируются только те узлы, которые изменяются, и все узлы на пути от них к корню; остальные узлы разделяются между версиями. Для сбалансированных деревьев затраты составляют $O(\log n)$ на операцию.
- **«Толстые» узлы** (fat node). Каждое поле узла хранит историю своих значений с метками версий. Изменение стоит $O(1)$ памяти, но доступ к полю в нужной версии требует поиска по истории — $O(\log m)$, где m — число версий.
- **Копирование узлов** (node copying, Дрисколл и др.). Комбинация двух предыдущих методов, дающая амортизированно $O(1)$ дополнительной памяти и времени на изменение для структур с ограниченной степенью входа узлов.

Для стека метод копирования пути вырождается в наилучший возможный случай: изменяется только вершина, а «путь от изменённого узла до корня» имеет длину 1 — это и есть сама вершина. Поэтому персистентный стек достигает тех же асимптотических оценок, что и эфемерный, без каких-либо дополнительных затрат.

1.6. Персистентная очередь

Тема курсовой работы допускает реализацию как стека, так и очереди, поэтому кратко рассмотрим, почему персистентная очередь устроена сложнее. Очередь работает по принципу FIFO: элементы добавляются в конец, а извлекаются из начала, т. е. изменения происходят на *двух* концах последовательности, и простое разделение хвоста, как в стеке, уже не работает.

Классическая эфемерная реализация очереди на двух стеках (входном и выходном) даёт амортизированное время $O(1)$. Однако при полной персистентности амортизационный анализ перестаёт быть справедливым: можно многократно применять «дорогую» операцию (перекладывание всего входного стека в выходной) к одной и той же старой версии, и каждый раз она будет стоить $\Theta(n)$. Для получения гарантированного $O(1)$ нужны более сложные схемы — очередь Худа—Мелвилла (real-time queue) или очереди К. Окасаки, использующие ленивые вычисления.

Существует и простой метод со сложностью O(log n), опирающийся на ту же идею дерева узлов. Все добавленные элементы хранятся в дереве, как в персистентном стеке (новый узел ссылается на предыдущий конец очереди), а версия очереди описывается парой (tail, len) — ссылкой на узел последнего элемента и длиной. Тогда push добавляет узел за O(1), pop уменьшает len на единицу за O(1), а элемент в начале очереди — это предок узла tail, расположенный на len − 1 уровней выше. Такого предка можно найти методом двоичного подъёма (binary lifting) за O(log n), храня в каждом узле ссылки на предков на расстоянии 2^j:

$$up_j(u) = up_{j-1}\left(up_{j-1}(u)\right), \quad up_0(u) = u.next$$

Это требует O(log n) дополнительной памяти на узел. В данной работе реализуется персистентный стек, так как он достигает оптимальных оценок O(1) по всем операциям при простой и наглядной реализации; персистентная очередь рассматривается как направление развития.

1.7. Асимптотический анализ

Для оценки сложности используются стандартные асимптотические обозначения. Для функций f(n), g(n) ≥ 0:

- f(n) = O(g(n)), если существуют c > 0 и n_0, такие что f(n) ≤ c·g(n) при всех n ≥ n_0 (верхняя оценка);
- f(n) = Ω(g(n)), если существуют c > 0 и n_0, такие что f(n) ≥ c·g(n) при всех n ≥ n_0 (нижняя оценка);
- f(n) = Θ(g(n)), если одновременно f(n) = O(g(n)) и f(n) = Ω(g(n)) (точная оценка).

Обозначения: n — число элементов в версии, к которой применяется операция; m — общее число выполненных операций (число версий); k — число элементов, которые требуется вывести.

Время выполнения операций. Каждая из операций push, pop, top, size выполняет фиксированное число действий (обращение к массиву roots по индексу, создание не более одного узла, чтение полей), не зависящее ни от n, ни от m. Следовательно, время каждой операции — Θ(1) в худшем случае (а не амортизированно). Нижняя оценка Ω(1) тривиальна для любой операции, поэтому полученные оценки оптимальны.

Операция	**Время**	**Доп. память**	**Комментарий**
push(v, x)	Θ(1)	Θ(1)	один новый узел + элемент roots
pop(v)	Θ(1)	Θ(1)	только элемент roots, узел не создаётся
top(v), size(v)	Θ(1)	—	чтение полей узла-вершины
доступ к любой версии	Θ(1)	—	индекс в массиве roots
вывод содержимого версии	Θ(k)	—	обход пути до корня
m произвольных операций	Θ(m)	Θ(m)	суммарно для всех версий

Память. Каждая операция push создаёт ровно один узел, а каждая операция (любого вида) добавляет одну ссылку в массив roots. Поэтому после m операций суммарный объём памяти составляет Θ(m), при этом одновременно доступны все m + 1 версий.

Сравнение с наивной персистентностью. Если перед каждой операцией копировать стек целиком, то i-я операция над стеком размера n_i стоит $\Theta(n_i)$. В худшем случае (m последовательных операций push) суммарные затраты времени и памяти составляют

$$\sum_{i=1}^{m} i = \frac{m(m+1)}{2} = \Theta(m^2)$$

против Θ(m) у персистентного стека. Например, при $m = 10^5$ наивный подход хранит порядка $5 \cdot 10^9$ элементов, что не помещается в оперативную память, тогда как персистентный стек — около 10^5 узлов.

Сравнение с журналом отката. Альтернативный способ возврата к прошлым состояниям — хранить эфемерный стек и журнал обратных операций (undo-log). Откат на d шагов стоит Θ(d), а переход к версии из другой ветки требует отката до общего предка и повторного применения операций, т. е. до Θ(m) времени. Кроме того, такой подход не позволяет одновременно читать несколько версий. Персистентный стек обеспечивает доступ к любой версии за Θ(1).

Высота дерева версий. Глубина узла равна размеру соответствующего стека, поэтому высота дерева версий не превосходит максимального размера стека и не влияет на сложность операций: ни одна операция не проходит по дереву больше чем на один уровень. Обход дерева нужен только для вывода содержимого версии (Θ(k)).

Освобождение памяти. Узел, на который не ссылается ни одна версия и ни один другой узел, становится недостижимым. В языках со сборкой мусора (Python, Java) такие узлы освобождаются автоматически; в C++ для этого используется подсчёт ссылок (std::shared_ptr). В рамках курсовой работы все версии хранятся до конца работы программы, поэтому узлы не удаляются.

1.8. Сравнение с альтернативными структурами

Структура	**push / pop**	**Доступ к старой версии**	**Память на операцию**	**Достоинства**	**Недостатки**
Эфемерный стек (массив)	O(1)*	невозможен	O(1)	простота, высокая скорость	старые версии теряются
Эфемерный стек + журнал отката	O(1)	O(d) — O(m)	O(1)	малый расход памяти	нет одновременного доступа к версиям, медленное переключение веток
Полное копирование	O(n)	O(1)	O(n)	универсальность	квадратичный расход памяти
Персистентный массив / дерево отрезков	O(log n)	O(1)	O(log n)	доступ к любому элементу по индексу	сложнее, медленнее в log n раз
«Толстые» узлы (fat node)	O(log m)	O(log m)	O(1)	общий метод для любых структур	поиск по истории версий

Структура	push / pop	Доступ к старой версии	Память на операцию	Достоинства	Недостатки
Персистентный стек (список)	O(1)	O(1)	O(1)	оптимальные оценки, простота	доступ только к вершине; k-й элемент — за O(k)

— амортизированная оценка с учётом расширения динамического массива

Из таблицы видно, что для задач, где нужны только операции стека (добавление и удаление с вершины), персистентный стек на неизменяемом списке является оптимальным решением: он не уступает эфемерному стеку по асимптотике ни одной операции, но дополнительно сохраняет все версии. Если же требуется произвольный доступ к элементам по индексу, следует использовать персистентный массив или персистентное дерево отрезков ценой множителя O(log n).

1.9. Примеры задач, решаемых с помощью выбранной структуры

1. **Отмена и повтор действий (undo/redo) с ветвлением.** В текстовых и графических редакторах каждое действие порождает новую версию стека состояний; возврат к любой точке истории и создание новой ветки выполняются за O(1). На этой идее основано «дерево отмены» (undo tree) в редакторах Vim и Emacs.
2. **Системы контроля версий.** История коммитов в Git устроена как неизменяемый связный список с разделением общих частей: каждый коммит ссылается на родителя, а ветки — это указатели на «вершины», как версии в персистентном стеке.
3. **Перебор с возвратом и поиск в пространстве состояний.** В алгоритмах поиска с возвратом (backtracking), при обходе игровых деревьев и в интерпретаторах логических языков (Prolog) персистентный стек позволяет хранить текущий путь решения без копирования при каждом ветвлении.
4. **Стек вызовов и окружения в интерпретаторах функциональных языков.** Неизменяемые списки являются базовой структурой данных в Lisp, Haskell, OCaml, Clojure; замыкания разделяют общие части окружения.
5. **Олимпиадные задачи на откаты и дерево версий:**

- Codeforces 707D «Persistent Bookcase» — https://codeforces.com/problemset/problem/707/D — операции над состоянием с возвратом к версии после k-й операции; решается построением дерева версий, совпадающего по устройству с деревом узлов персистентного стека;
- Codeforces 1858E1 «Rollbacks (Easy Version)» — https://codeforces.com/problemset/problem/1858/E1 — массив с операциями «добавить в конец», «удалить k последних элементов» и «откатить последнее изменение»; это стековые операции с откатом версий;
- Codeforces 1858E2 «Rollbacks (Hard Version)» — https://codeforces.com/problemset/problem/1858/E2 — та же задача в онлайн-режиме, где персистентное хранение версий становится необходимым.

На платформах acmp.ru и informatics.msk.ru задачи непосредственно на персистентный стек отсутствуют. В качестве смежных можно использовать задачи раздела «Стек» на informatics.msk.ru (реализация стека, проверка правильности скобочной последовательности), дополнив их требованием хранить и восстанавливать предыдущие состояния стека.

Источники к главе 1

1. Driscoll J. R., Sarnak N., Sleator D. D., Tarjan R. E. Making data structures persistent // Journal of Computer and System Sciences. — 1989. — Vol. 38, № 1. — P. 86–124.
2. Okasaki C. Purely Functional Data Structures. — Cambridge University Press, 1998.
3. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2013.
4. Hood R., Melville R. Real-time queue operations in pure LISP // Information Processing Letters. — 1981. — Vol. 13, № 2. — P. 50–54.
5. Кнут Д. Э. Искусство программирования. Т. 1. Основные алгоритмы. — 3-е изд. — М.: Вильямс, 2000.
6. Персистентный стек // Викиконспекты ИТМО: neerc.ifmo.ru/wiki.
7. Demaine E. Advanced Data Structures. Lecture 1: Persistent Data Structures // MIT OpenCourseWare 6.851: ocw.mit.edu.